Integrating Blockchain and IPFS for Traceability, Integrity, and Ex-Post Auditing in Federated Learning

David Piotr Znojek[1*] and Francisco A. Pujol[1]

[1*]Department of Computer Technology and Computation, University of Alicante, Carretera de San Vicente del Raspeig s/n, San Vicente del Raspeig, 03690, Alicante, Spain.

*Corresponding author(s). E-mail(s): dawidznojek@gmail.com;
Contributing authors: fpujol@ua.es;

Abstract

Federated learning preserves edge data privacy but lacks intrinsic mechanisms to verify participant contributions, exposing distributed systems to poisoning attacks. This paper presents a hybrid architecture integrating a permissioned Ethereum Virtual Machine blockchain (Hyperledger Besu) with off-chain content-addressable storage (InterPlanetary File System). A smart contract coordinates bidirectional cross-verification using cryptographic commitments of model weights, complemented by an independent ex-post audit protocol to mathematically verify aggregation fidelity. Evaluated on a distributed testbed across 15 executions (450 rounds) classifying vehicular road events, the blockchain layer adds an overhead of only 9.5% (6.34 seconds per round) with deterministic, invariant gas consumption. In-transit tampering is intercepted within 149 milliseconds, and ex-post auditing reconstructs canonical aggregation in 0.34 seconds with exact mathematical correspondence. These findings confirm that the operational cost of decentralization, immutability, and non-repudiation is thoroughly viable and affordable compared to the critical auditability guarantees obtained.

Keywords: Federated learning, Blockchain, IPFS, Traceability, Ex-post auditing, Byzantine fault tolerance

1 Introduction

The rapid proliferation of Artificial Intelligence (AI) and Machine Learning (ML) has transformed critical sectors, including healthcare, finance, and intelligent transportation systems. However, training high-capacity models requires massive volumes of data, which frequently contain sensitive or proprietary information, such as high-resolution geolocation traces, user behavior patterns, or biometric identifiers. Under traditional centralized paradigms, gathering raw data into a single repository introduces severe privacy vulnerabilities, regulatory compliance challenges (such as the GDPR), and substantial computational and communication bottlenecks.

Federated Learning (FL) [1] emerged as a decentralized paradigm where distributed edge nodes iteratively train local model instances on their private datasets, transmitting only updated weight vectors or gradients to a central coordinator. Although FL avoids direct exposure of raw data, it introduces a structural security challenge: the absence of inherent trust among participating entities. In an unsupervised, distributed environment, malicious actors or compromised edge nodes can launch data poisoning, model poisoning, or backdoor insertion attacks [2] by submitting manipulated model updates. For instance, in Connected and Autonomous Vehicles (CAVs) [3], a subtle, coordinated perturbation in model parameters could lead the global classifier to misidentify critical road hazards or traffic signals (e.g., failing to detect a STOP sign), yielding catastrophic consequences.

To address these vulnerabilities, integrating Blockchain technology with Federated Learning (BCFL) [4] provides an immutable, decentralized, and verifiable ledger [5, 6]. By embedding a distributed ledger into the federated training workflow, parameter updates, aggregation rounds, and participant identities are cryptographically anchored via smart contracts, establishing an end-to-end, auditable, and non-repudiable record of all training contributions.

Nevertheless, this convergence introduces a critical system-level trade-off: **security versus operational performance**. While blockchain provides immutability and Byzantine fault tolerance, it inherently introduces network latency, consensus delays, transaction confirmation overhead, and additional computational resource demands. In mission-critical edge computing environments [7], it is essential to answer a fundamental question: *what temporal and computational overhead must be incurred to achieve a more auditable and tamper-resistant federated training?*

1.1 Motivation and Prior Work

This work builds upon and extends two research lines conducted at the University of Alicante:

1. **Real-time vehicular event verification** [8]: demonstrated the feasibility of a private Hyperledger Besu network operating under Istanbul Byzantine Fault Tolerant (IBFT 2.0) consensus to log vehicular traffic alerts with sub-second latencies (0.32 to 0.82 s), outperforming human reaction times.
2. **FL-Blockchain architectural framework** [9]: conceptually proposed using smart contracts to record cryptographic hashes of model weights at each federated round.

The present study formally designs, implements, and empirically validates the convergence of these proposals on a physical distributed testbed, evaluating both cryptographic integrity guarantees and temporal system performance.

1.2 Main Contributions

The main contributions of this paper are summarized as follows:

- **Hybrid architecture design**: We formulate an end-to-end framework coupling a permissioned Hyperledger Besu blockchain network with the Flower federated learning framework [10] and off-chain, content-addressable distributed storage via IPFS.
- **Traceability and integrity via smart contracts**: We design and deploy a Solidity smart contract that orchestrates bidirectional cross-verification of models through content identifiers (IPFS CIDs, encapsulating SHA-256 multihashes of serialized artifacts), immutably anchoring each participant's contribution and the aggregated global model per round.
- **Empirical adversarial validation**: We subject the platform to deliberate weight-corruption attack vectors (both in-transit tampering from an untrusted server and unauthorized/poisoning injections from clients), validating that the bidirectional verification protocol deterministically intercepts integrity discrepancies and rogue models with low response latencies (31 to 149 ms).
- **Ex-post audit protocol and mathematical FedAvg verification**: We develop and evaluate an independent, off-band audit protocol enabling any consortium participant or third-party auditor to retrospectively verify aggregator fidelity. We demonstrate the ability to mathematically reconstruct canonical FedAvg aggregation directly from immutable commitments stored on Besu and IPFS, achieving exact numerical correspondence ($\Delta = 0$, CID match) in 0.34 s, while detecting unauthorized parameter deviations in 0.18 s.
- **Quantitative infrastructure benchmarking**: We deploy a convolutional traffic event classifier to serve as a representative workload for extracting reproducible system-level metrics (temporal overhead, EVM gas consumption, and validator scaling), delimiting hyperparameter and model optimization as future work.
- **Operational feasibility analysis**: We analytically resolve the latency-versus-security dilemma, characterizing the operational regimes where blockchain logging overhead is thoroughly justified compared to unverified setups.

To facilitate scientific reproducibility and technology transfer, the complete framework source code, Solidity smart contracts, Hyperledger Besu Docker Compose deployment templates, and experimental evaluation scripts are openly available on GitHub: https://github.com/dpz6-ua/Blockchain_ML.

1.3 Paper Organization

The remainder of this paper is structured as follows: Section 2 reviews related work and theoretical foundations of Blockchain and Federated Learning. Section 3 details

the proposed architecture and smart contract design. Section 4 describes the experimental testbed and dataset. Section 5 presents and discusses empirical performance, scalability, and security validation results. Finally, Section 6 concludes the paper and outlines future research directions.

2 Theoretical Foundations and Related Work

The proposed architecture converges three fundamental technological pillars: permissioned distributed ledgers with Byzantine Fault Tolerant (BFT) consensus, content-addressable distributed storage, and federated learning. Below, we synthesize their theoretical foundations and examine the critical limitations identified in the related literature.

2.1 Permissioned Blockchain and IBFT 2.0 Consensus

A blockchain is an immutable, distributed data structure where transactions are bundled into cryptographically linked blocks using collision-resistant hash functions (such as SHA-256) [6, 11]. Unlike public (permissionless) networks such as Bitcoin or Ethereum Mainnet, which suffer from high transaction latency and volatile monetary fees, industrial consortia predominantly adopt permissioned networks compatible with the Ethereum Virtual Machine (EVM) [12], such as Hyperledger Besu [13].

In these environments, compute-intensive (Proof-of-Work, PoW) or stake-based (Proof-of-Stake, PoS) mechanisms are replaced by Byzantine Fault Tolerant consensus algorithms. The **IBFT 2.0** (*Istanbul Byzantine Fault Tolerant*) protocol [14] organizes consensus around a predefined set of validator nodes that alternate the proposer role in a round-robin schedule. A proposed block is appended to the ledger only after a three-phase voting process (pre-prepare, prepare, commit) signed by a two-thirds supermajority of validators:

$$n \geq 3f + 1 \tag{1}$$

where n denotes the total number of validator nodes and f represents the maximum number of tolerated faulty or Byzantine participants.

IBFT 2.0 provides a pivotal advantage over probabilistic consensus models: **immediate deterministic finality**. Because the chain does not experience probabilistic forks, once a training round commitment or model update is sealed within a block, it achieves immediate finality and permanent immutability on the ledger. Furthermore, while the transaction gas price is set to zero in private consortia to eliminate monetary friction, the computational metric of *gas used* [15] remains an objective, deterministic measure of execution and storage overhead imposed on the EVM.

2.2 Off-Chain Distributed Storage (IPFS)

Directly persisting deep neural network parameters on a blockchain ledger is technically infeasible due to model artifact sizes (ranging from dozens to hundreds of megabytes), which would rapidly trigger severe EVM state bloat and demand prohibitive gas execution limits.

To overcome this bottleneck, we adopt the paradigm of **off-chain storage with on-chain commitment** utilizing the InterPlanetary File System (IPFS) [16, 17]. IPFS operates over a peer-to-peer (P2P) network based on content addressing and Merkle Directed Acyclic Graph (Merkle DAG) structures [17, 18]: files are partitioned into immutable chunks and addressed through a unique Content Identifier (CID) derived directly from their cryptographic hash. The global-scale empirical and architectural evaluation conducted by Trautwein et al. [18] demonstrates that its block-exchange protocol (Bitswap) and distributed hash table (Kademlia DHT) achieve decentralized content routing with minimal overhead. By decoupling weight artifact storage to IPFS and recording only the content identifier (IPFS CIDv0, encapsulating the SHA-256 multihash of the serialized weights) within the smart contract [19], subsequent downloads can be validated against the immutable ledger. This provides data deduplication, formal integrity verification, and distributed access at negligible on-chain execution cost, provided that operational availability and persistence are backed by content pinning or dedicated IPFS nodes.

2.3 Federated Learning and the FedAvg Algorithm

Federated Learning (FL) [1] preserves data privacy by decoupling local model optimization from central parameter aggregation. Among federated consensus mechanisms, **FedAvg** (*Federated Averaging*) serves as the foundational standard. At each communication round t, the aggregation coordinator broadcasts the current global weight vector $w^{(t)}$ to a selected set of K edge clients. Each client trains the model on its private partition $\mathcal{D}_k$ by minimizing a local loss function, yielding updated local parameter weights $w_k^{(t)}$. The server then computes the consolidated global model as a sample-weighted average based on local sample counts n_k:

$$w^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} w_k^{(t)}, \quad \text{where } n = \sum_{k=1}^{K} n_k \tag{2}$$

The **Flower** framework [10] offers a modular, efficient infrastructure to coordinate this federated lifecycle via bidirectional remote procedure calls over gRPC.

2.4 Vulnerabilities in FL and Related Work

Despite privacy advantages, conventional Federated Learning frameworks suffer from intrinsic vulnerabilities classified in recent security taxonomies and surveys [4, 27, 28]:

1. **Central server as a single point of failure and trust**: Existing frameworks offer no native mechanism to verify whether the central coordinator faithfully executes FedAvg or intentionally injects malicious biases into the global parameter vector $w^{(t+1)}$.
2. **Poisoning attacks and lack of non-repudiation**: Compromised edge clients can execute model poisoning or inject stealthy backdoors [2]. In the absence of an immutable, non-repudiable log linking cryptographic identities to parameter

Table 1 Qualitative and architectural comparison against relevant Blockchain-FL and robustness frameworks in the literature.

Proposal / Framework	**Blockchain Consensus**	**Model Storage**	**Verification Protocol**	**Transactional Cost**	**Fault Tolerance**
Krum (Blanchard et al.) [20]	No Blockchain (Centralized)	Central server (SPOF)	Geometric selection (L_2)	None (No ledger)	Strict Byzantine ($f < \frac{N-2}{2}$)
BlockFL (Kim et al.) [21]	PoW (Slow / Inefficient)	On-Chain (Storage bottleneck)	Unidirectional	Volatile cryptocurrency	Probabilistic ($< 50\%$)
DeepChain (Weng et al.) [22]	PoW / PBFT with incentives	On-Chain (Encrypted gradients)	Homomorphic Enc. + MACs	Severe (Massive on-chain ciphertext)	Formal Byzantine ($f < N/3$)
AV-BFL (Pokhrel & Choi) [23]	Adaptive PoW / DPoS	On-Chain at RSUs	Base station verification	Wireless retransmissions	Constrained by vehicular channel
H-BCFL (Chai et al.) [24]	Hierarchical DPoS at RSUs	On-Chain at base stations	Reputation and knowledge audit	Edge clustering overhead	Dependent on RSU reliability
B5G-FL (Lu et al.) [25]	Permissioned (PBFT / BFT)	On-Chain at base stations	Base station consensus	DRL optimization	Formal Byzantine ($f < N/3$)
Vehicular BFL (Pujol et al.) [8]	IBFT 2.0 (Permissioned)	IoT alert logging	Traffic monitoring	Commercial gas-free	Formal Byzantine ($N \geq 3f + 1$)
BCFL-Ref (Goh et al.) [26]	PoW / PoA (Ethereum)	Hybrid Off-Chain (IPFS)	Unidirectional (No cross-verification)	Variable gas (Public/private net)	Formal Byzantine (Ethereum consensus)
Our Architecture	**QBFT / IBFT 2.0 (Besu)**	**Content-addressable storage (IPFS)**	**Bidirectional Cross-Verif. + Ex-Post Audit**	**Stable gas per transaction**	**Formal Byzantine ($f = 1 \rightarrow 2$)**

digests, attributing malicious updates or auditing the history of compromised models becomes impossible.

To address these vulnerabilities, researchers have increasingly investigated the hybridization of Blockchain and Federated Learning (BCFL) [4]. However, designing practical architectures for Connected and Autonomous Vehicles (CAVs/IoV) involves four fundamental engineering trade-offs:

1. On-Chain vs. Hybrid Off-Chain Storage and Computation: Pioneer architectures such as BlockFL [21] proposed exchanging full weight matrices directly over the blockchain ledger. Similarly, DeepChain [22] integrated decentralized auditing and economic incentives with homomorphic encryption (HE) of gradients, incurring extreme computational and on-chain storage bottlenecks. Other designs, such as FLChain [29], explored decentralized auditing using distributed Bloom filters, finding that serializing massive parameter tensors within blockchain transactions triggers acute EVM state bloat and degrades throughput. Consequently, the hybrid paradigm—leveraging content-addressable off-chain storage via IPFS anchored to on-chain cryptographic hashes [19, 26]—represents an efficient approach to prevent ledger bloat while ensuring scalability in distributed deployments.

2. Consensus Mechanisms and Latency in Wireless and Vehicular Networks: In autonomous driving and edge computing (5G and beyond), communication latency and resource constraints are primary bottlenecks [23, 25]. Proof-of-Work (PoW) protocols introduce multi-minute confirmation delays and probabilistic forks that are intolerable in safety-critical vehicular systems [21]. To optimize resource allocation and model convergence in cellular edge networks, Lu et al. [25] formulated a base-station permissioned blockchain utilizing deep reinforcement learning (DRL) for dynamic bandwidth allocation, while Pokhrel and Choi [23] developed analytical models characterizing vehicular wireless channel dynamics on consensus delay. In the Internet of Vehicles (IoV), Chai et al. [24] proposed a hierarchical architecture offloading aggregation and DPoS consensus to Roadside Units (RSUs), Qi et al. [30] combined permissioned blockchain with differential privacy for traffic flow prediction, and Abou El Houda et al. [31] devised collaborative intrusion detection schemes via BCFL. However, these frameworks depend heavily on the continuous availability and synchronization of fixed roadside infrastructure. Rather than relying on high-latency public networks or foregoing Byzantine fault tolerance, permissioned consortia running IBFT 2.0 / QBFT such as Hyperledger Besu [8] provide immediate deterministic finality and rigorous Byzantine resilience ($N \geq 3f + 1$) without monetary fee fluctuations.

3. Cryptographic Integrity vs. Statistical Robustness: Two complementary defensive tiers exist in federated learning security [27, 28]. On the one hand, decentralized immutable ledgers provide formal guarantees regarding in-transit artifact integrity, sender authentication, and non-repudiation. On the other hand, if a participating client deliberately manipulates its local training data while legitimately signing and registering the resulting weights on-chain (*data/model poisoning*), statistical aggregation defenses against anomalous gradients are essential. Literature has explored geometric filtering such as Krum [20], coordinate-wise trimming including Trimmed-Mean and Median [32], and hybrid defenses against complex

collusions such as Bulyan [33]. Similarly, FLTrust [34] introduced trust bootstrapping by evaluating angular parameter similarity against a root dataset maintained by the coordinator. These two layers are complementary: on-chain non-repudiation unambiguously attributes each parameter update to a specific cryptographic identity, enabling post-facto forensic auditing and smart-contract-coordinated slashing once statistical anomalies are identified.

4. Aggregator Accountability (Black-Box Aggregation vs. Ex-Post Auditing): In verifiable aggregation literature without blockchain, pioneering works such as VerifyNet [35] and VeriFL [36] address verifiable aggregation using homomorphic commitments, zero-knowledge proofs, and secret sharing schemes. Although these mechanisms formally ensure that the central coordinator computes aggregation faithfully without reconstructing raw gradients, they impose heavy computational overhead and synchronous communication delays on edge clients during every federated round. Conversely, most BCFL architectures either assume the central coordinator is trustworthy or attempt to execute aggregation directly within the EVM at prohibitive gas and computational costs. To the best of our knowledge, existing frameworks lack a decoupled, deterministic mechanism that—without penalizing operational cycle times ($< 0.08\%$ overhead)—empowers any consortium participant or external auditor to reconstruct canonical FedAvg *ex-post* from immutable Blockchain and IPFS commitments, mathematically verifying global model fidelity without compromising local data privacy.

To contextualize the contributions of this research with respect to prior work, Table 1 presents a qualitative and architectural comparison. Unlike purely centralized statistical schemes (e.g., Krum), PoW-based architectures or massive on-chain storage paradigms (BlockFL, DeepChain), frameworks reliant on fixed RSU infrastructure, or unidirectional verification models lacking cross-validation (Goh et al. [26]), our architecture implements bidirectional cross-verification paired with an independent ex-post audit protocol over a permissioned consortium blockchain with immediate finality and stable gas costs per transaction.

3 Proposed System Architecture

To provide federated learning with robust traceability and Byzantine fault resilience, we propose a hybrid architecture decoupled into three operational planes [18, 19]: the federated computation plane (Flower), the off-chain content-addressable storage plane (IPFS), and the on-chain consensus and immutability plane (Hyperledger Besu). Figure 1 illustrates the end-to-end interaction among these components.

3.1 Smart Contract Design: FLRegistry

Governance and audit logic are encapsulated within the `FLRegistry` smart contract, implemented in Solidity (compiler version `0.8.10`). Its primary objective is to maintain a deterministic, chronological audit log of each federated round without storing massive numerical parameter matrices on-chain.

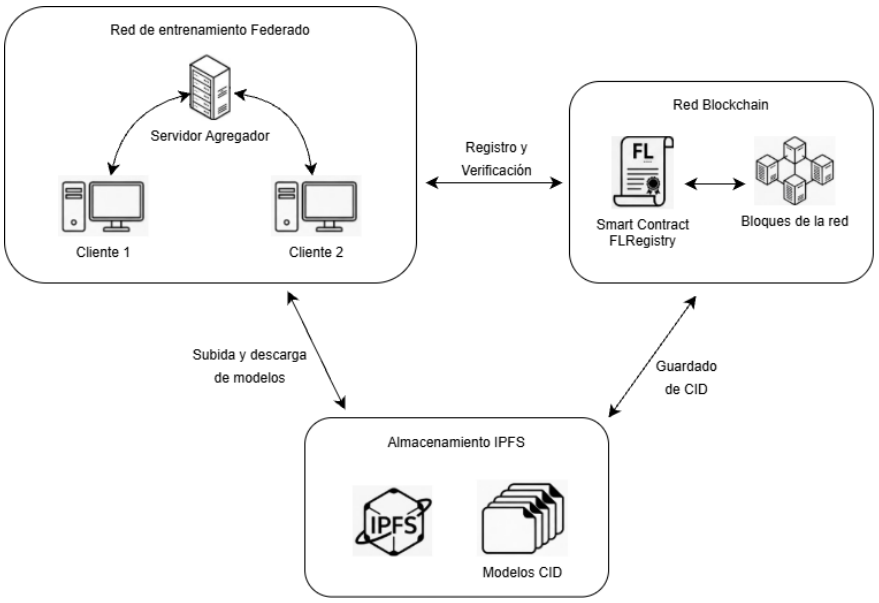


Fig. 1 Overall hybrid system architecture: decoupling between federated computation (Flower), off-chain parameter storage (IPFS), and immutable traceability logging (Hyperledger Besu with IBFT 2.0).

3.1.1 Data Structures and Persistence

The core entity within the contract is the `ModelRound` structure, defined as:

```
struct ModelRound {
    uint256 roundNum;
    string modelHash;
    address author;
    uint256 timestamp;
}
```

where `roundNum` indexes the federated iteration, `modelHash` stores the IPFS Content Identifier (CIDv0, encapsulating the SHA-256 multihash of the serialized model), `author` records the Ethereum public address of the sender (`msg.sender`), and `timestamp` captures the block validation timestamp.

Contract state is organized through two complementary mappings:

- `mapping(uint256 => ModelRound) public history`: sequentially records the global models aggregated by the coordinator at the conclusion of each round t.
- `mapping(uint256 => mapping(address => ModelRound)) public clientHistory`: indexes the individual local contributions submitted by each client in each round, enabling granular forensic auditing.

Additionally, state pointers `latestRound` and boolean flag `hasModel` are maintained to facilitate direct and constant-time read access without requiring iterative storage scans.

3.1.2 State-Modifying Primitives and Access Control

The contract exposes two state-modifying transactional functions that emit the decoupled `ModelUpdated` event:

- `registerClientModel(uint256 round, string memory hash)`: Invoked by each edge client k upon completing local training, immutably binding the CID of its contribution to its Ethereum address (`msg.sender`).
- `updateModel(uint256 round, string memory hash)`: Invoked operationally by the aggregation coordinator after executing the aggregation algorithm. To enforce consortium governance and prevent unauthorized participants or rogue nodes from submitting fraudulent global models, this function is guarded by the `onlyServer` access control modifier:

```
modifier onlyServer() {
    require(msg.sender == serverAddress,
            "Unauthorized sender");
    _;
}
```

  where `serverAddress` is initialized in the contract constructor with the authorized coordinator public address (which can be reassigned or governed via consortium multi-signature schemes). Consequently, the EVM deterministically reverts any unauthorized call to `updateModel`.

3.1.3 Read and Verification Primitives

To avoid imposing execution costs during routine checks, model verification queries are implemented as `view` functions, resolved locally at the JSON-RPC node without consuming gas:

- `getLastModel()`: Enables any client to retrieve the CID of the latest consolidated global model along with its validation timestamp.
- `getClientModel(uint256 round, address client)`: Enables the coordinator and external auditors to cross-check the CID reported by a client against its signed on-chain record.

3.1.4 Model-Agnostic Design and Generalization

The proposed framework is architecturally and operationally agnostic to neural network topology, data modality, and training framework. Because `FLRegistry` processes exclusively uniform 46-character string identifiers (the IPFS CIDs encapsulating the multihash digest of the serialized weights), EVM gas costs remain predictable and entirely decoupled from model parameter dimensions—whether the weights represent

a shallow classifier, a deep convolutional network, or a Transformer architecture. Similarly, the IPFS storage plane treats serialized parameters as generic byte payloads, making this bidirectional verification protocol directly applicable across diverse modalities (tabular data, time series, acoustic signals, or computer vision) and federated optimization algorithms.

3.2 Operational Workflow and Bidirectional Cross-Verification

The federated lifecycle consists of a one-time bootstrap phase followed by an iterative protocol featuring strict bidirectional verification, detailed below:

3.2.1 Phase 0: Bootstrap and Baseline Model (Round 0)

Prior to concurrent training execution:

1. The Hyperledger Besu network operating under IBFT 2.0 consensus and the IPFS daemon are initialized.
2. `FLRegistry` is deployed, and its contract address and ABI are published in a shared configuration manifest.
3. The coordinator instantiates the initial model parameters $w^{(0)}$, serializes them, and uploads them to IPFS, obtaining the initial identifier CID_0.
4. The coordinator executes `updateModel(0,` CID_0`)`, establishing an auditable cryptographic baseline for all participants.

3.2.2 Phase 1: Downward Verification (Client Verifies Server)

Upon receiving the round start signal and candidate weights $w_{cand}^{(t)}$ via gRPC (Figure 2):

1. The client queries the blockchain by calling `getLastModel()`, retrieving the officially committed $\text{CID}_{reg}^{(t-1)}$.
2. The client fetches the referenced model artifact from IPFS and performs an exhaustive, element-wise numerical comparison across all tensor layers against $w_{cand}^{(t)}$ (verifying parametric equality within a floating-point tolerance of $\epsilon = 10^{-7}$).
3. If any discrepancy is detected between the delivered weights and the on-chain commitment, the client **aborts training immediately**, effectively neutralizing split-view attacks where a compromised coordinator attempts to distribute biased models selectively to specific clients.

3.2.3 Phase 2: Local Training and Contribution Commitment

If the downward verification check passes successfully (Figure 3):

1. The client performs local training on its private data partition.
2. It serializes the updated local weight vector $w_k^{(t)}$ and uploads it to IPFS, obtaining $\text{CID}_k^{(t)}$.
3. It broadcasts a signed transaction calling `registerClientModel(`t, $\text{CID}_k^{(t)}$`)` and awaits confirmation within an IBFT 2.0 block receipt.

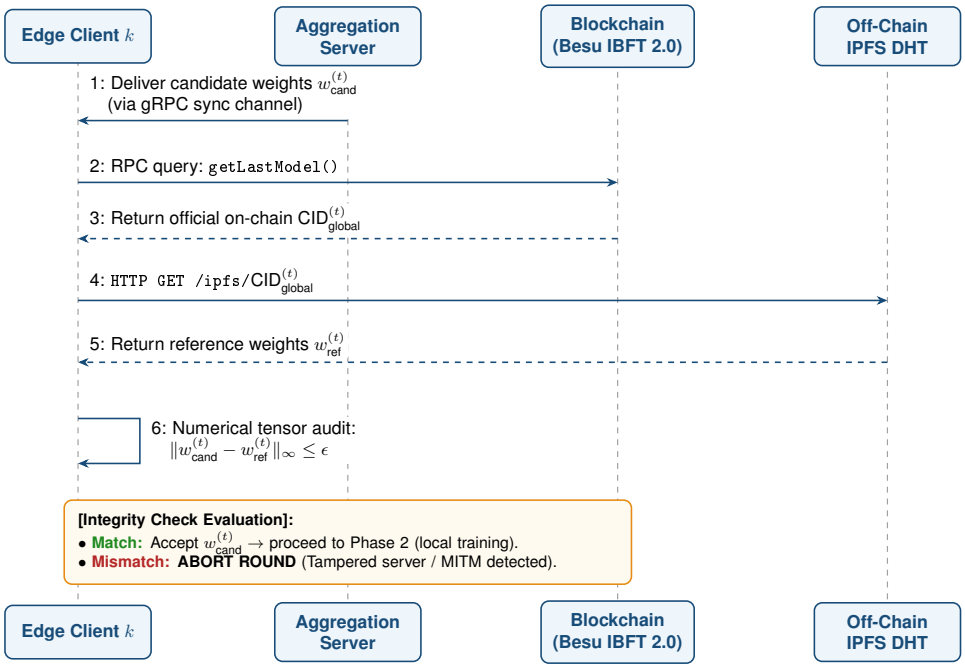


Fig. 2 Phase 1: Downward verification at the edge client: validating received candidate weights against the on-chain registered CID retrieved from IPFS.

4. It returns the parameter payload along with $\text{CID}_k^{(t)}$ and its public key to the Flower coordinator.

3.2.4 Phase 3: Upward Verification and Global Aggregation

Upon receiving client updates at the aggregation coordinator (Figure 4):

1. For each client k, the server receives parameter payload $w_k^{(t)}$, identifier $\text{CID}_{k,\text{gRPC}}^{(t)}$, and client address address_k.
2. **In-memory integrity verification:** The server deterministically computes the cryptographic digest of the received weight vector: $\text{CID}_{k,\text{calc}}^{(t)} = \text{Hash}(w_k^{(t)})$.
3. **On-chain verification and double cross-check:** The server queries `getClientModel(`t`, address`$_k$`)` from `FLRegistry`, retrieving $\text{CID}_{k,\text{onchain}}^{(t)}$, and enforces strict double equality:

$$\text{CID}_{k,\text{calc}}^{(t)} == \text{CID}_{k,\text{gRPC}}^{(t)} == \text{CID}_{k,\text{onchain}}^{(t)} \tag{3}$$

This dual check counters two complementary threat vectors: if a malicious client commits a valid CID on Besu but transmits tampered weights over gRPC, or if a network adversary alters parameters in transit, $\text{CID}_{k,\text{calc}}^{(t)} \neq \text{CID}_{k,\text{onchain}}^{(t)}$; similarly, if the gRPC-reported CID does not match the signed transaction on Besu,

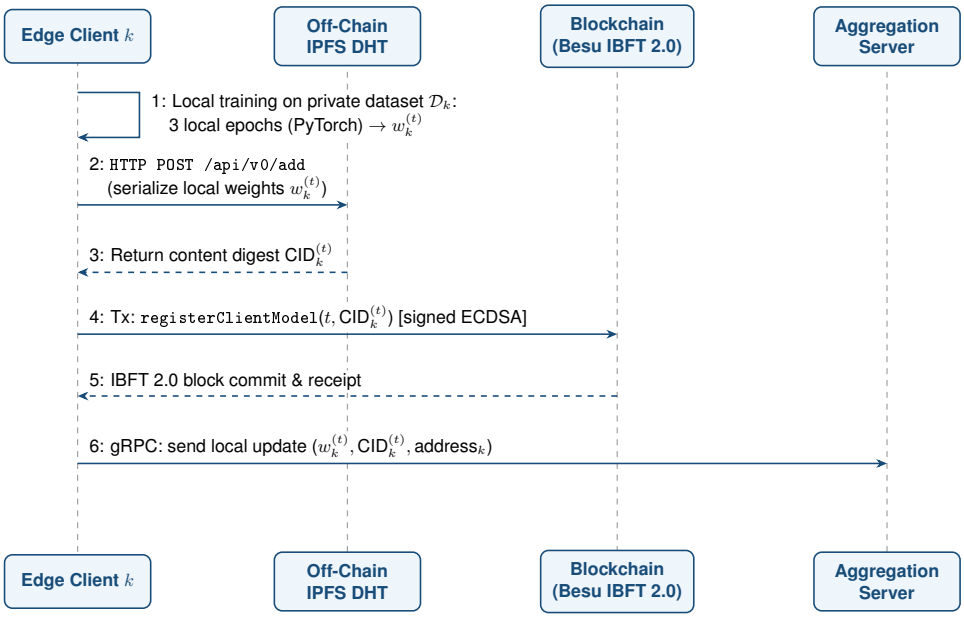


Fig. 3 Phase 2: Local training at the client, parameter upload to IPFS, and signed transaction commitment to Hyperledger Besu.

verification fails immediately. In either case, the contribution is discarded prior to aggregation.

4. The server executes FedAvg aggregation exclusively over verified contributions:

$$w^{(t+1)} = \sum_{k \in \mathcal{K}_{valid}} \frac{n_k}{n_{valid}} w_k^{(t)} \tag{4}$$

where n_k is the local sample count of client k and $n_{valid} = \sum_{k \in \mathcal{K}_{valid}} n_k$.

5. The server uploads the aggregated model to IPFS, obtaining $\text{CID}_{global}^{(t+1)}$, and anchors it on Besu by executing `updateModel`$(t+1, \text{CID}_{global}^{(t+1)})$. Because `updateModel` is restricted by `onlyServer`, no outside party can overwrite the consolidated global model.

3.3 Ex-Post Auditing and Verifiability of FedAvg

A distinctive architectural capability enabled by our hybrid decoupling is the ability to deterministically verify coordinator honesty retrospectively (*ex-post*). In conventional federated setups, the aggregation server operates as a trusted black box: participants must trust that aggregated weights $w^{(t+1)}$ faithfully represent a weighted average over received contributions, without arbitrary omissions, unauthorized weight injections, or stealthy backdoors.

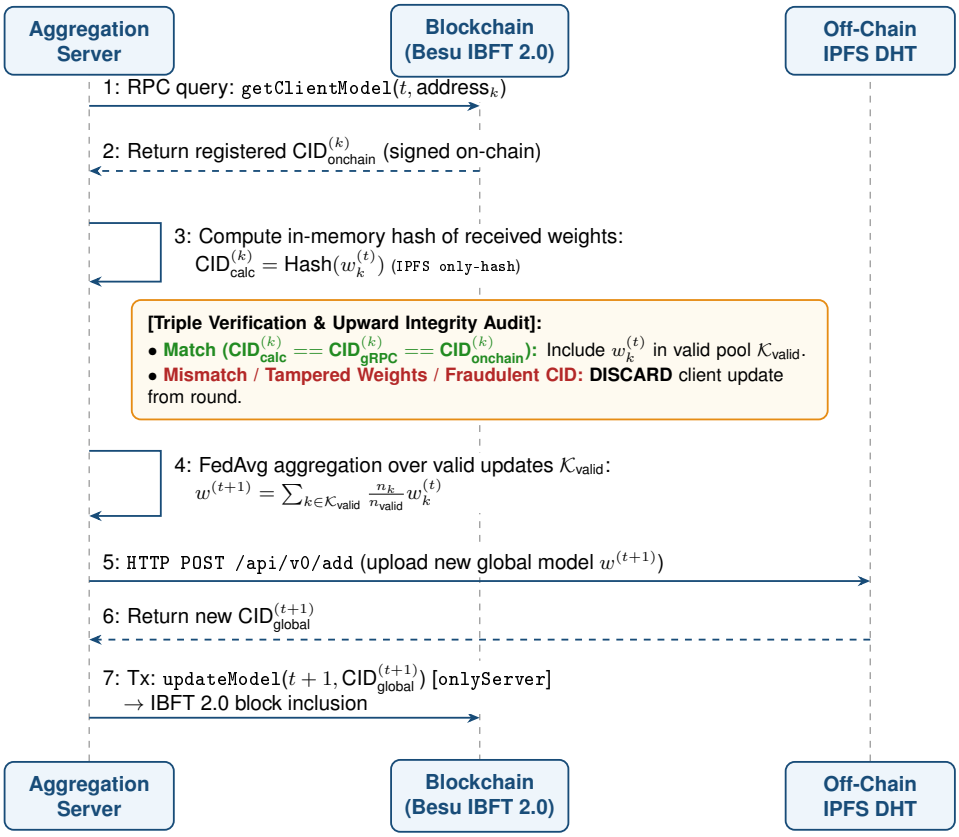


Fig. 4 Phase 3: Upward verification at the aggregation coordinator: tensor integrity validation, on-chain cross-check, FedAvg aggregation, and global round commitment.

In our decoupled architecture, every round t produces an immutable and reproducible audit trail linking the blockchain ledger to the content repository:

1. **Participant identification $\mathcal{K}_t$:** Because Solidity `mapping` structures are non-iterable by EVM design, active participants in round t are discovered deterministically by querying `ModelUpdated(round, modelHash, author)` event logs emitted during `registerClientModel` calls via standard RPC methods (`eth_getLogs`), or from the permissioned consortium whitelist.
2. **Local weight commitments:** Each participating client $k \in \mathcal{K}_t$ signs and registers its $\text{CID}_k^{(t)}$ in `clientHistory`, accessible via `getClientModel(`t`, address`$_k$`)`.
3. **Sample weighting factors n_k:** In balanced, homogeneous deployments (such as the evaluated experimental benchmark, where clients operate over equivalent IID partitions, $n_k = N/|\mathcal{K}_t|$), weighting factors are symmetric ($n_k / \sum n_j = 1/|\mathcal{K}_t|$). In heterogeneous settings (non-IID), scalar values n_k can be embedded within the IPFS metadata descriptor or included as an event parameter, allowing exact reconstruction without exposing raw training samples.

4. **Global model commitment:** The coordinator seals $\text{CID}_{\text{global}}^{(t+1)}$ in `history`, accessible via `getLastModel()` or direct round indexing.

From this immutable audit trail, any consortium member or independent third-party auditor can execute a deterministic aggregation check without accessing private datasets:

1. **Artifact retrieval:** The auditor queries Besu to retrieve active clients $\mathcal{K}_t$ and their committed $\text{CID}_k^{(t)}$, then downloads the binary parameter tensors $w_k^{(t)}$ from IPFS.
2. **Input integrity validation:** It verifies that for each client $k \in \mathcal{K}_t$, the cryptographic digest of the downloaded weights strictly matches the on-chain committed $\text{CID}_k^{(t)}$.
3. **Mathematical aggregation reconstruction:** It computes the canonical FedAvg aggregation:
$$\hat{w}^{(t+1)} = \sum_{k \in \mathcal{K}_t} \frac{n_k}{\sum_{j \in \mathcal{K}_t} n_j} w_k^{(t)} \tag{5}$$
4. **Global model cross-check:** It serializes $\hat{w}^{(t+1)}$ and computes its content identifier $\text{CID}_{audit}^{(t+1)} = \text{Hash}(\hat{w}^{(t+1)})$. Finally, it evaluates whether $\text{CID}_{audit}^{(t+1)} == \text{CID}_{global}^{(t+1)}$, registered in `FLRegistry`.

If $\text{CID}_{audit}^{(t+1)} \neq \text{CID}_{global}^{(t+1)}$, any coordinator deviation—whether due to biased weighting, participant omission, or unauthorized parameter tampering—is cryptographically, publicly, and non-repudiably proven. This verifiability transforms the centralized aggregator into a strictly accountable, provably honest orchestrator.

3.4 Threat Model and Security Scope

To clearly delineate the defensive guarantees provided by the framework, this section defines the threat model, distinguishing between vulnerabilities mitigated by design and those that lie outside our scope.

3.4.1 Mitigated Threats (In-Scope)

- **Man-in-the-Middle tampering**: A network adversary intercepting the gRPC channel who attempts to alter local parameter tensors sent to the server or global models delivered to clients. Discrepancies are intercepted immediately through element-wise numerical checks and on-chain CID verification.
- **Server equivocation / split-view attacks**: An untrusted or compromised coordinator attempting to deceive specific edge nodes by distributing divergent or altered global models (e.g., to facilitate active gradient inversion or localized performance degradation, as formalized by Carletti et al. [37]). Because clients accept only weights whose content identifier strictly matches the authoritative commitment in `FLRegistry`, divergent distribution attempts are blocked locally.
- **Unauthorized global model injection**: Attempts by rogue participants or external adversaries to commit unauthorized global models to the blockchain are rejected by the EVM through the `onlyServer` modifier.

- **Contribution repudiation**: Elliptic curve digital signatures (ECDSA secp256k1) underpinning Hyperledger Besu transactions prevent participants from repudiating submitted parameter updates.

3.4.2 Out-of-Scope Threats and System Boundaries

- **Semantic data and model poisoning with valid signatures**: If an adversary trains on poisoned datasets (e.g., label flipping or stealthy backdoors) but legitimately signs and registers the resulting model updates on Besu, cryptographic verification confirms provenance and integrity but cannot assess gradient statistical quality. Defense against semantic attacks requires pairing on-chain traceability with robust Byzantine aggregation rules (e.g., Krum [20], Median, Trimmed Mean) or forensic ex-post auditing of flagged contributions.
- **Private key compromise**: Cryptographic guarantees assume secure client key management. In industrial deployments, private keys must be secured via Hardware Security Modules (HSMs) or Trusted Execution Environments (TEEs).
- **Inference and gradient inversion attacks**: While our framework ensures integrity and provenance, it does not incorporate Differential Privacy (DP) or Secure Multiparty Computation (SMPC) to obfuscate shared gradients against reconstruction attacks [37].

3.4.3 Privacy and Weight Exposure on IPFS

On a public IPFS instance, any node knowing a model CID could retrieve parameter tensors and potentially attempt training data extraction. To protect model privacy in consortium deployments with stringent confidentiality requirements (e.g., autonomous fleets or healthcare), the framework supports two hardening mechanisms:

1. **Private IPFS Swarm**: Configuring the IPFS daemon in private swarm mode using a shared key (`swarm.key`) and setting `LIBP2P_FORCE_PNET=1`. This prevents unauthorized nodes from connecting to the P2P overlay or fetching storage blocks.
2. **Payload encryption at rest**: Encrypting serialized parameter vectors (e.g., using AES-256-GCM) prior to IPFS publication. Symmetric decryption keys can be distributed via asymmetric public-key cryptography to authorized consortium members or managed through Attribute-Based Encryption (ABE).

4 Implementation and Experimental Setup

To empirically isolate and quantitatively evaluate the temporal and computational overhead introduced by the traceability and auditability layer, we deployed an experimental testbed representative of a distributed vehicular computing environment.

4.1 Hardware and Software Infrastructure

The empirical evaluation was conducted using two physical workstations interconnected within the same local area network (LAN). Their technical specifications are detailed in Table 2:

- **Station 1 (Server, Validators, and IPFS)**: Hosts the Flower aggregation coordinator, the IPFS daemon, and all validator nodes of the permissioned Hyperledger Besu network (4 validators in configurations E1 and E4; 7 validators in E5). Each validator runs as an independent containerized service via Docker Compose, interconnected through a virtual bridge network that emulates the peer-to-peer consensus mesh of a vehicular consortium and exposes the JSON-RPC interface of the primary validator node as a gateway for the other entities.
- **Station 2 (Edge Nodes / Clients)**: Hosts and executes the federated training clients (members 2 and 3), which communicate with the aggregation server and the blockchain network over the physical LAN connection.

Table 2 Specifications of the experimental testbed.

Parameter	**Station 1 (Server/RPC)**	**Station 2 (Clients)**
Processor	Intel Core i5-1135G7	Intel Core i7-9700
Base frequency	2.40 GHz	3.00 GHz
Cores / Threads	4 cores / 8 threads	8 cores / 8 threads
RAM	7.6 GB (allocated in WSL2)	8.0 GB
Operating System	Ubuntu 22.04 LTS (WSL2)	Windows 11

The software stack comprises Hyperledger Besu operating under the QBFT / IBFT 2.0 consensus protocol with a block generation period fixed at 5 seconds (`blockperiodseconds: 5`), supporting private transaction processing via Tessera and orchestrated using Docker Compose; the IPFS Kubo daemon (`v0.32.1`); Python `3.12` with PyTorch [38] for neural model training and inference; Flower (`flwr`) [10] for federated gRPC communication; and the `web3.py` library for JSON-RPC connectivity with the EVM. All smart contracts, network configurations, and orchestration scripts are publicly accessible at https://github.com/dpz6-ua/Blockchain_ML.

The two workstations communicate over a 5 GHz Wi-Fi 802.11ac wireless local area network, exhibiting an average round-trip time (*RTT ping*) of 3.8 ± 1.1 ms and a sustained bandwidth of 100 Mbps. The serialized binary model artifact (`NetCliente`) has a size of exactly 1.95 MB. To ensure agile off-chain storage operations during iterative training, parameter uploads to IPFS are executed via the HTTP API endpoint `/api/v0/add?pin=false`, generating the CID identifier and adding the block to the local repository in milliseconds without the synchronous latency of pinning. We note that while `pin=false` optimizes immediate per-round latency, it does not inherently guarantee indefinite artifact persistence against future IPFS garbage collection sweeps; industrial production deployments would require explicit pinning policies or synchronization with a persistent IPFS cluster.

4.2 Neural Network Model and Vehicular Dataset

To simulate a real-world driver assistance system in Connected and Autonomous Vehicles (CAVs) [3], we adopted the road traffic event classification dataset provided by CloudLab-AIA [9].

The classification task addresses road hazard detection across four distinct categories:

1. Traffic accident (`accident`).
2. Heavy congestion / dense traffic (`dense_traffic`).
3. Adverse weather conditions (`meteorology`).
4. Smooth traffic flow (`sparse_traffic`).

The dataset comprises 3,415 images distributed across: adverse weather (1,027), smooth traffic (947), dense traffic (778), and accidents (663). Images are preprocessed by resizing to a resolution of 32×32 pixels across three RGB channels and normalized. To ensure statistical comparability and eliminate distributional bias, the dataset is evenly split across $K = 2$ clients following an Independent and Identically Distributed (IID) partition.

The implemented model (`NetCliente`) is a Convolutional Neural Network (CNN) structured into two consecutive convolutional blocks (with 3×3 kernels, ReLU activation, and Max Pooling subsampling), followed by two dense fully connected layers that project the feature vector to 4 output logits. Model training optimizes Cross-Entropy Loss using the Adam optimizer with a learning rate $\eta = 10^{-3}$, a batch size of 16 for training and 32 for validation, 3 local training epochs per federated round, and a total horizon of 30 federated rounds.

We emphasize that the primary objective of this study is not to design an optimal convolutional architecture or maximize classification accuracy, but rather to establish a realistic federated learning testbed to quantify and analyze system-level performance metrics (temporal overhead, scalability, and gas consumption). Model-level hyperparameter tuning, overfitting mitigation, and deeper architectures are deliberately left as future work, with the neural network serving here as a representative, realistic workload. Consequently, the decoupled architecture is directly transferable to other distributed learning domains (e.g., smart healthcare, smart grids, or collaborative finance), where blockchain audit logic remains identical and only IPFS off-chain transfer and hash computation scale with serialized model volume.

4.3 Experimental Configuration Matrix

To decouple and analytically quantify the individual impact of each architectural component, five experimental configurations (E1 to E5) were evaluated, as summarized in Table 3.

This experimental design enables controlled single-variable comparisons:

- **Direct Blockchain overhead**: Comparing E1 vs. E2 in a single-machine setup, and E4 vs. E3 in a distributed network setup.
- **Effect of physical distribution**: Comparing E2 vs. E3 (without Blockchain) and E1 vs. E4 (with a 4-validator Blockchain).

Table 3 Matrix of evaluated experimental configurations.

Exp. ID	Workstations	Blockchain Layer	Validators	Objective
E1	1	Yes	4	Local BC overhead
E2	1	No	—	Local baseline
E3	2	No	—	Distributed baseline
E4	2	Yes	4	Distributed BC overhead
E5	2	Yes	7	IBFT 2.0 scalability

- **BFT consensus scalability**: Comparing E4 (4 validators) vs. E5 (7 validators), evaluating latency overhead when elevating Byzantine fault tolerance from $f = 1$ to $f = 2$.

5 Experimental Evaluation and Results

This section quantitatively analyzes the performance of the proposed architecture across the experimental testbed, directly addressing the core research questions: the temporal overhead of the auditability layer, the impact of physical resource decoupling, the scalability of the IBFT 2.0 Byzantine consensus, the empirical resilience against adversarial tampering attacks, and the predictability of EVM gas consumption. To ensure statistical soundness and experimental reproducibility, each of the five experimental configurations (E_1 to E_5) was evaluated across 3 complete, independent executions of 30 federated rounds each, yielding a corpus of 15 executions and 450 total federated rounds ($N_{\text{total}} = 450$). We note that the model convergence trajectory is identical with or without the blockchain layer, as the traceability framework operates fully decoupled and does not modify training tensors or gradients.

5.1 Mathematical Formalization of Performance Metrics

To isolate the latency imposed by each system component, the total round time at the edge nodes (T_{client}) and at the central coordinator (T_{server}) during iteration t is analytically defined as:

$$T_{\text{client}}^{(t)} = t_{\text{verif_desc}}^{(t)} + t_{\text{train}}^{(t)} + t_{\text{ipfs_up}}^{(t)} + t_{\text{tx_besu}}^{(t)} \tag{6}$$

$$T_{\text{server}}^{(t)} = t_{\text{wait}}^{(t)} + t_{\text{verif_asc}}^{(t)} + t_{\text{fedavg}}^{(t)} + t_{\text{ipfs_global}}^{(t)} + t_{\text{block_confirm}}^{(t)} \tag{7}$$

where $t_{\text{verif_desc}}$ denotes local downward verification against the Besu CID; t_{train}, gradient computation time in PyTorch; $t_{\text{ipfs_up}}$, binary model publication to IPFS; $t_{\text{tx_besu}}$, transaction issuance and confirmation for `registerClientModel`; t_{wait}, the coordinator gRPC reception window; $t_{\text{verif_asc}}$, on-chain validation via `getClientModel`; t_{fedavg}, matrix aggregation; and $t_{\text{block_confirm}}$, IBFT 2.0 block confirmation following `updateModel`.

These two timing perspectives provide complementary system insights:

- **Client cycle time (T_{client}, reported in Table 4):** Quantifies local computation and communication duration at edge nodes (vehicles), from the receipt of the global model to the confirmation of their contribution on the blockchain. This metric governs client operational autonomy and maintains comparability with prior benchmarks [9].
- **End-to-end round time at the server (T_{server}, broken down in Table 5):** Encompasses the entire round orchestrated by the central coordinator, including gRPC client synchronization and block inclusion of the aggregated global model via `updateModel` ($\approx$ 2.8–5.1 s additional latency per round).

Relative temporal overhead (Δ_{overhead}) between a secure configuration E_{BC} and its corresponding baseline E_{base} is calculated using the distribution medians:

$$\Delta_{\text{overhead}} = \frac{\text{Med}(T_{E_{\text{BC}}}) - \text{Med}(T_{E_{\text{base}}})}{\text{Med}(T_{E_{\text{base}}})} \times 100 \quad [\%] \tag{8}$$

Regarding statistical methodology, the experimental protocol treats the **15 complete executions** as independent sample units ($N_{\text{runs}} = 3$ independent repetitions per configuration). The 30 consecutive rounds of each run are not treated as independent and identically distributed (i.i.d.) observations, because round $t+1$ depends intrinsically on the model state and system conditions at round t. Consequently, all 450 rounds (90 per configuration) are utilized for longitudinal descriptive analysis and trajectory visualization (Figure 5), while inter-run reproducibility is quantified via the Standard Error (SE) across execution means ($SE = \sigma_{\text{runs}}/\sqrt{N_{\text{runs}}}$).

5.2 Consolidated Temporal Performance Overview

Table 4 summarizes the per-round client cycle time (T_{client}) across the 15 independent runs (3 per configuration, totaling 90 rounds per environment), reporting the median, mean, inter-run standard deviation (σ_{runs}), and standard error ($SE = \sigma_{\text{runs}}/\sqrt{3}$). Figure 5 depicts the continuous time-series trajectories across all rounds.

Table 4 Consolidated per-round client cycle time (T_{client}) across 15 independent executions (3 per configuration, 30 rounds/run). Values of σ_{runs} and SE quantify dispersion and standard error across the 3 independent execution means ($SE = \sigma_{\text{runs}}/\sqrt{3}$).

Exp.	Nodes	Configuration	Runs	Median (s)	Mean (s)	σ_{runs} (s)	SE (s)
E_1	1	4 val · with BC	3	**85.42**	97.09	20.01	11.55
E_2	1	Baseline without BC	3	**75.70**	76.45	1.65	0.95
E_3	2	Baseline without BC	3	**66.62**	67.62	2.29	1.32
E_4	2	4 val · with BC	3	**72.96**	73.29	1.28	0.74
E_5	2	7 val · with BC	3	**73.35**	74.40	0.54	0.31

The multi-run analysis reveals significant differences in operational stability across environments. In the local single-workstation configuration E_1, inter-run variability

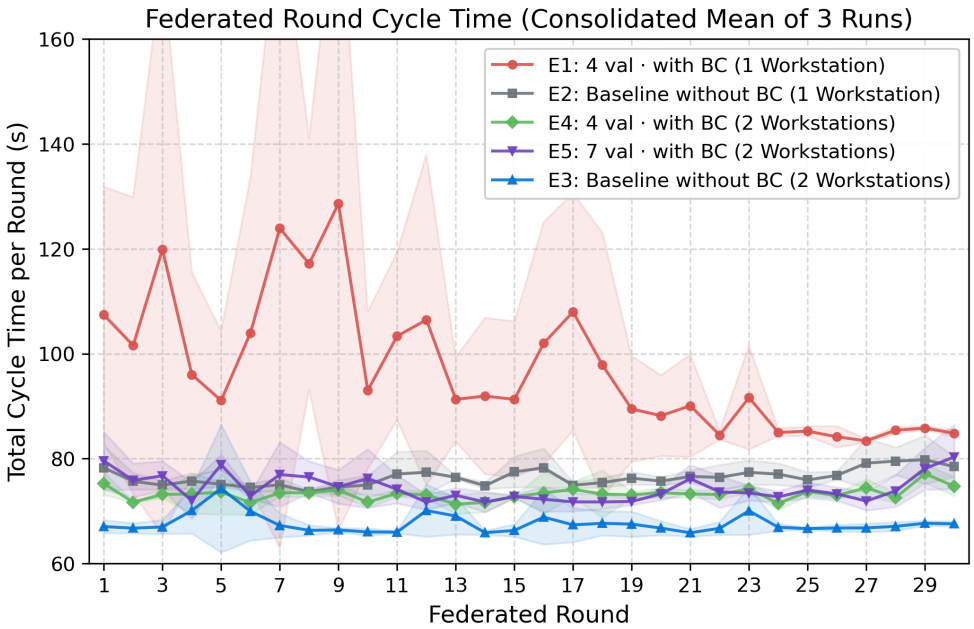


Fig. 5 Comparison of client cycle time (T_{client}) across federated rounds for the five evaluated configurations.

across execution means reached $\sigma_{\text{runs}} = 20.01$ s ($SE = 11.55$ s), with an aggregated longitudinal standard deviation across all 90 rounds of $\sigma_{\text{pool}} = 27.64$ s. This pronounced dispersion occurred because, during single-machine execution, local convolutional training competed directly for CPU and memory resources with eight background Docker containers hosting the Besu validator nodes and the IPFS daemon. Conversely, in the distributed deployment (E_4 and E_5), inter-run variability dropped markedly ($\sigma_{\text{runs}} \in [0.54, 1.28]$ s and $SE \leq 0.74$ s; with pooled longitudinal standard deviations constrained to $\sigma_{\text{pool}} \leq 3.92$ s), demonstrating excellent operational stability. Consequently, the **median** is adopted as the primary metric to isolate intrinsic system performance from transient external perturbations.

5.3 Isolated Temporal Overhead of the Blockchain Layer

To evaluate the precise temporal cost incurred by tensor serialization, IPFS upload, ECDSA signing, and IBFT 2.0 consensus, we compare corresponding configurations with and without the security layer (Figure 6):

- **Single-Workstation Scenario (E_1 vs. E_2)**: Median cycle time increases from 75.70 s to 85.42 s, representing an overhead of **+9.72 s (+12.8%)**.
- **Distributed Scenario (E_3 vs. E_4)**: Median cycle time increases from 66.62 s to 72.96 s, corresponding to an overhead of only **+6.34 s (+9.5%)**.

This finding is of particular significance for vehicular and edge systems: when offloading client training to dedicated hardware, the latency impact of the auditability

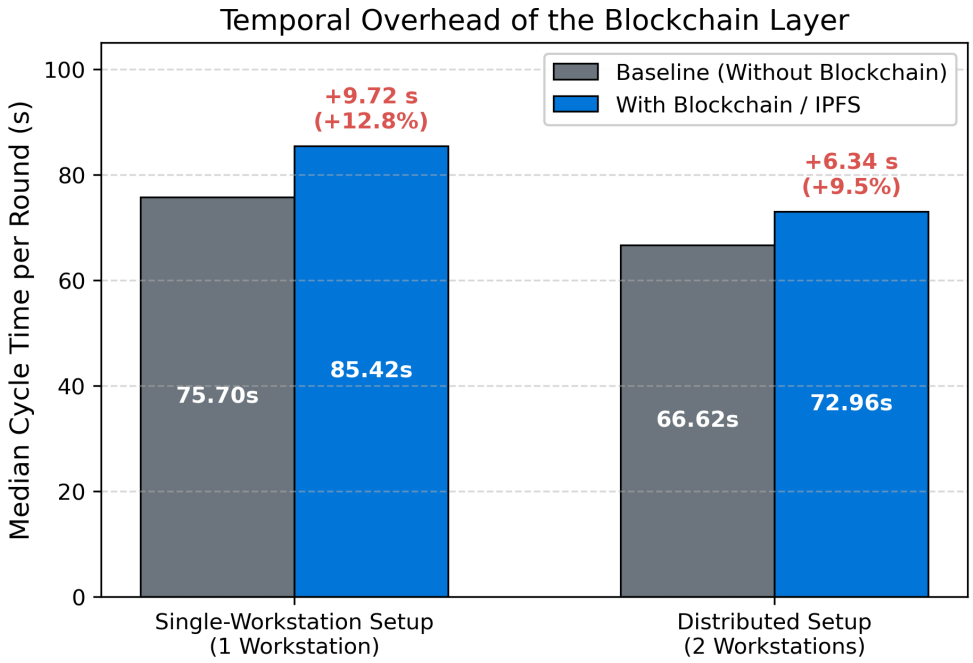


Fig. 6 Temporal overhead induced by blockchain integration in single-workstation and distributed deployments.

layer remains strictly below 10%. An additional cost of approximately 6 seconds per round represents a practical, viable trade-off in exchange for end-to-end immutability, non-repudiation, and verifiable integrity.

5.4 Physical Distribution Effect and Phase Breakdown

Comparing single-workstation and distributed configurations confirms that physical role decoupling significantly improves overall throughput (Figure 7): round latency decreases both in the baseline setup without blockchain (from 75.70 s to 66.62 s, a 12.0% speedup) and with blockchain (from 85.42 s to 72.96 s, a 14.6% speedup).

To analyze the underlying factors driving this behavior, Table 5 provides a fine-grained breakdown of average latency across individual phases at both clients (Station 2) and server (Station 1), demonstrating strict correspondence with the aggregate metrics in Table 4.

The empirical breakdown in Table 5 substantiates three fundamental principles:

1. **Computation decoupling:** In distributed configurations (E_3, E_4, E_5), time dedicated exclusively to PyTorch backpropagation remains constant ($\sim$ 65.4–67.6 s), eliminating the CPU contention observed in E_1 (94.08 s).
2. **Bounded network and verification overhead:** Remote JSON-RPC verification (1.68–1.91 s at clients, $\sim$ 5 ms at coordinator), IPFS network transfer (2.43–3.93 s), and in-memory multihash calculation at the coordinator ($\sim$ 52 ms in final code,

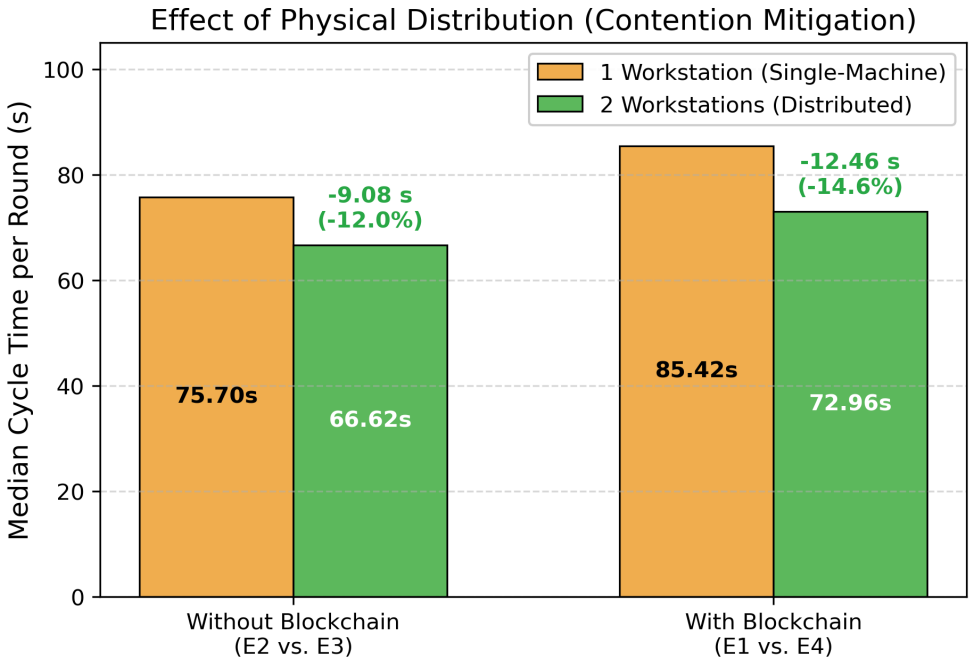


Fig. 7 Effect of physical node distribution on cycle time: mitigating CPU/memory contention.

Table 5 Detailed breakdown of average phase latencies for clients (T_{client}) and coordinator (T_{server}) (in seconds).

	Client Phases (Station 2)				Server Phases (Station 1)			
Exp.	**Train**	**Verif.**	**IPFS**	**Tx Besu**	**Wait**	**Verif.**	**FedAvg**	**Block**
E_1	94.08	0.08	0.06	2.85	98.16	0.05	0.0065	5.14
E_2	76.45	0.00	0.00	0.00	77.02	0.00	0.0040	0.00
E_3	67.62	0.00	0.00	0.00	71.07	0.00	0.0036	0.00
E_4	66.22	1.68	2.43	2.84	79.54	0.06	0.0047	2.89
E_5	65.40	1.91	3.93	2.94	82.63	0.06	0.0079	2.99

$< 0.08\%$ of the cycle) introduce minimal overhead, while FedAvg aggregation is practically instantaneous (< 8 ms).

3. **Consistency between T_{client} and T_{server}:** The sum of the four client-side local phases (*Train* + *Verif.* + *IPFS* + *Tx Besu*) strictly accounts for the client cycle time in Table 4 (e.g., in E_4: $66.22 + 1.68 + 2.43 + 2.84 = 73.17$ s, matching the empirical mean of 73.29 s). Concurrently, server perspective reflects overall round duration (T_{server}), where the coordinator awaits concurrent client completion (*Wait*, 79.54 s) and commits the aggregated global model on-chain (*Block*, 2.89 s), totaling an end-to-end latency of ≈ 82.49 s.

5.5 IBFT 2.0 Byzantine Consensus Scalability (4 vs. 7 Validators)

In permissioned consortium networks, Byzantine fault tolerance requires $N \geq 3f + 1$, where N is the total validator count and f is the maximum number of tolerated faulty or adversarial nodes:

- 4-validator network (E_4): tolerates $f = 1$ faulty node; signature quorum $2f + 1 = 3$.
- 7-validator network (E_5): tolerates $f = 2$ faulty nodes; signature quorum $2f + 1 = 5$.

Comparing E_4 against E_5, median cycle time per round exhibits only a slight change, rising from 72.96 s to 73.35 s (+0.39 s), demonstrating that round cadence remains dominated by local model training rather than consensus messaging.

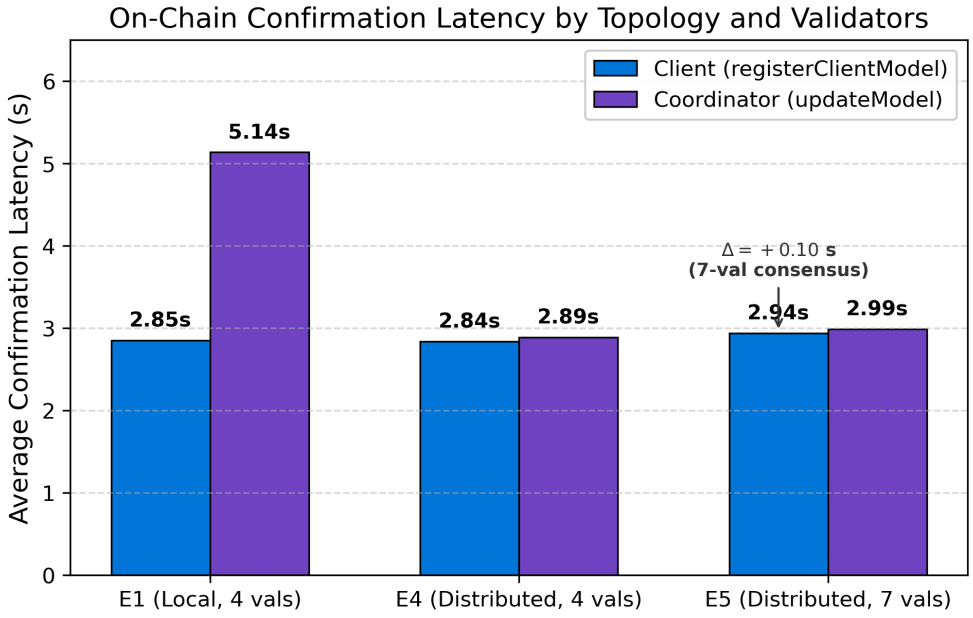


Fig. 8 Average on-chain transaction confirmation latency by topology and validator count in Hyperledger Besu.

Isolating **on-chain transaction confirmation latency** (Figure 8) reveals the empirical impact of expanding the consensus committee from 4 to 7 validator nodes in the evaluated testbed:

- Client transaction (`registerClientModel`): mean confirmation latency increases from 2.84 s in E_4 to 2.94 s in E_5 (+0.10 s).
- Coordinator transaction (`updateModel`): mean confirmation latency increases from 2.89 s in E_4 to 2.99 s in E_5 (+0.10 s).

This symmetric 100 ms increase reflects the empirical consensus communication overhead observed when expanding the validator quorum ($f = 1 \rightarrow 2$). This indicates that elevated Byzantine resilience can be readily accommodated without disrupting training throughput.

5.6 Adversarial Robustness and Experimental Attack Verification

To empirically verify that the architecture actively intercepts real-world adversarial vectors rather than merely logging nominal transactions, controlled attack scenarios were executed via the `test_verificacion.py` test suite. The evaluation assesses system behavior against the two primary Byzantine threats in untrusted federated environments: coordinator equivocation or split-view attacks (downward flow) and poisoning client updates (upward flow).

5.6.1 Test 1: Downward Verification (Tampered Server vs. Edge Client)

In untrusted federated environments, distributing global weights exposes edge clients to two primary risks: in-transit parameter tampering by network intermediaries (*Man-in-the-Middle*) and malicious server equivocation, where a compromised coordinator delivers divergent or deliberately poisoned models to specific participants. Crucially, the validity of the on-chain global model is strictly governed by the `onlyServer` access modifier in `FLRegistry`, which deterministically reverts unauthorized attempts to record global models. Consequently, the client-side downward verification routine autonomously safeguards edge nodes against discrepancies between gRPC-delivered weights and the authoritative blockchain commitment:

1. **Generation and Authoritative Commitment:** The legitimate coordinator instantiates the `NetCliente` baseline model, serializes weights using `pickle.dumps`, and publishes them to the local IPFS daemon at `/api/v0/add?pin=false`, obtaining CID `QmQTpgejdzh...`. The coordinator then broadcasts a signed transaction calling `updateModel(round, official_cid)` from its authorized address (`0xfe3b55...`), sealing the cryptographic commitment in **Block 80**.
2. **Adversarial Injection and Discrepancy Simulation:** An attack scenario is simulated by injecting additive perturbation into parameters delivered to the client ($\mathbf{w}_{\text{cand}} = \mathbf{w}_{\text{official}} + 0.5$) and transmitting the perturbed vector via gRPC (modeling in-transit alteration or an equivocation attempt).
3. **Cross-Verification and Immediate Abort:** Upon receiving the training request, the client queries `getLastModel()` on the smart contract via JSON-RPC to retrieve the on-chain committed CID. It concurrently downloads the reference binary artifact from the IPFS gateway (`/ipfs/<CID>`) and performs an element-wise numerical tensor comparison within floating-point tolerance (`np.allclose(p_received, p_ipfs, atol=1e-7)`).

Algorithm 1. Downward Verification at the Edge Client

1. **Input:** Candidate weights $\mathbf{w}_{\text{cand}}$ received via gRPC from coordinator.
2. Query Besu smart contract: $\text{CID}_{\text{onchain}} \leftarrow$ `FLRegistry.getLastModel().cid`
3. Fetch binary model from IPFS: $\mathbf{w}_{\text{ipfs}} \leftarrow$ `IPFS.get`$(\text{CID}_{\text{onchain}})$
4. **If** $\exists l : \|\mathbf{w}_{\text{cand},l} - \mathbf{w}_{\text{ipfs},l}\|_\infty > \epsilon$ **then**
   - Emit alert: *"Discrepancy detected in received weights"*
   - **ABORT** local round immediately (block backpropagation)
5. **Else:** Accept $\mathbf{w}_{\text{cand}}$ and proceed with local training.

Figure 9 shows runtime console telemetry at the edge client. The complete verification sequence (RPC query to Besu, IPFS download, and tensor comparison) required only **0.1486 seconds** (149 ms). Upon detecting the discrepancy, the client immediately terminated execution, preventing local model corruption and safeguarding private vehicular training data.

```
================================================================================
TEST 1: DOWNWARD INTEGRITY VERIFICATION (Malicious Server -> Client Aborts)
================================================================================
1. Uploading official legitimate global model to IPFS...
   Official CID generated: QmQTpgejdzhKCA83ZHLVAKE9m2dKhLS92RqkQY5ziztvMV
2. Registering official CID in FLRegistry.sol (updateModel)...
   Official CID sealed on-chain in Block 80 (Tx: 652b4fceef...)
3. Malicious server delivers tampered weights to edge client via gRPC...
4. Client queries Blockchain getLastModel() and downloads IPFS reference...
   Audit execution latency: 0.1486 seconds
   Bit-by-bit tensor weight comparison: DISCREPANCY DETECTED
   -> [EXPERIMENTAL SUCCESS] Client intercepted server-side tampering.
   -> Action: ABORT TRAINING ROUND (Private edge dataset safeguarded).
```

Fig. 9 Test 1: Console log of edge client detecting parameter tampering against the official on-chain CID sealed in block 80.

Figure 10 shows the immutable blockchain evidence captured from the Hyperledger Besu block explorer. It documents the confirmation of transaction `updateModel` in **Block 80**, submitted by coordinator address `0xfe3b557e...` with transaction hash `0x973b0b...` and gas consumption of 167,460 in this preliminary run (which becomes 169,649 in the final contract featuring `onlyServer`, gas limit 500,000). This bijective correspondence between the mined block and client verification confirms protocol consistency.

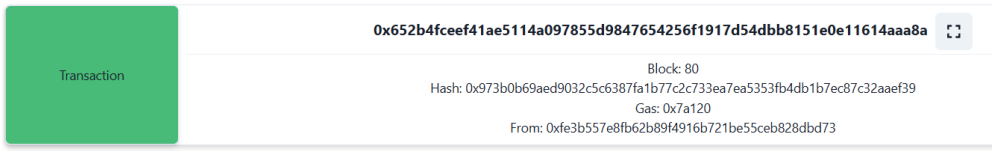


Fig. 10 Test 1: Confirmation of Block 80 in Hyperledger Besu, recording the authoritative `updateModel` transaction submitted by the coordinator.

5.6.2 Test 2: Upward Verification (Poisoning Client vs. Aggregation Coordinator)

The second critical vulnerability is the injection of corrupt weights or gradients by a compromised client (*model poisoning*). To intercept this at the gateway, the coordinator cross-verifies the bijective correspondence between received memory payloads and signed on-chain commitments:

1. **Legitimate On-Chain Registration:** The honest client node (account 3, address `0xf17f52...`) uploads its local weights to IPFS and submits a signed transaction calling `registerClientModel(round, legitimate_cid)`, confirmed in **Block 81**.
2. **Adversarial Injection:** An injection attempt is simulated where an update with altered parameters or mismatched metadata is delivered to the coordinator (modeling the transmission of fraudulent CID `QmClientFraud...` or poisoned parameters that deviate from the registered CID).
3. **Cryptographic Audit and Exclusion from FedAvg:** Prior to executing FedAvg, the coordinator computes the cryptographic hash of received memory tensors ($\text{CID}^{(k)}_{\text{calc}}$) and executes a read-only JSON-RPC call `getClientModel(round, address)` against `FLRegistry` to retrieve $\text{CID}^{(k)}_{\text{onchain}}$. Upon detecting discrepancy, the update is excluded before aggregation.

Algorithm 2. Upward Cryptographic Audit and RPC Check at Coordinator

1. **Input:** Weights $\mathbf{w}_k^{(t)}$ and identifier $\text{CID}^{(k)}_{\text{rep}}$ reported by client k with address addr_k.
2. Compute hash digest of received in-memory weights: $\text{CID}^{(k)}_{\text{calc}} \leftarrow \text{Hash}(\mathbf{w}_k^{(t)})$
3. Query client on-chain record from Besu: $\text{CID}^{(k)}_{\text{onchain}} \leftarrow \texttt{getClientModel}(t, \text{addr}_k)$
4. **If** $\text{CID}^{(k)}_{\text{rep}} \neq \text{CID}^{(k)}_{\text{onchain}}$ **or** $\text{CID}^{(k)}_{\text{calc}} \neq \text{CID}^{(k)}_{\text{onchain}}$ **then**
   - Emit alert: *"Discrepancy detected / Unauthenticated injection"*
   - **DISCARD** client k contribution from FedAvg pool
5. **Else:** Include weights $\mathbf{w}_k^{(t)}$ in global aggregation.

Figure 11 illustrates server terminal output. The JSON-RPC verification call to the EVM required only **0.0309 seconds** (31 ms). Confirming that the submitted CID did not match the client signed transaction, the server discarded the rogue update immediately, preserving global model integrity.

Figure 12 displays the transaction receipt in **Block 81** from the Besu explorer, documenting the transaction `registerClientModel` issued by client account `0xf17f5215...` with hash `0x347899f...` and gas consumption of 160,200 (gas limit 200,000). Because the adversary transmitted a CID divergent from that sealed on-chain, the cross-check intercepted the injection due to the collision resistance of the underlying SHA-256 multihash.

```
================================================================================
TEST 2: UPWARD INTEGRITY AUDIT (Malicious Client -> Server Discards from FedAvg)
================================================================================
1. Honest Client (0xf17f5215...) registers on-chain...
   Legitimate CID sealed on-chain in Block 81 (Tx: 47d2bfbb2c...)
2. Malicious client attempts injecting fraudulent CID to server via gRPC:
   Reported payload CID: QmClientFraudNotRegisteredInBesu22222222
3. Aggregation server audits reported CID via getClientModel(round, address)...
   RPC query latency:       0.0309 seconds
   On-chain registered CID: QmClientLegitModelWeightsCID1111111111111111
   ECDSA signature & Hash match: NO (FRAUD DETECTED)
   -> [EXPERIMENTAL SUCCESS] Aggregator intercepted poisoning attempt.
   -> Action: DISCARD CLIENT CONTRIBUTION from FedAvg pool.
   -> Global model convergence protected; Byzantine resilience verified.
```

Fig. 11 Test 2: Coordinator console log auditing client CID via RPC and discarding fraudulent contribution prior to aggregation.

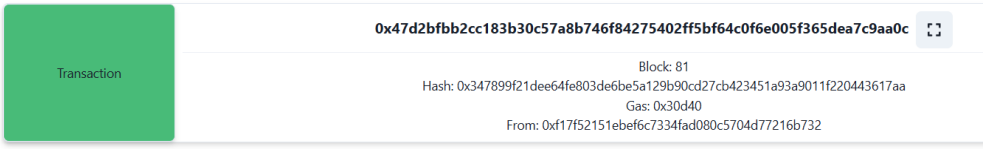


Fig. 12 Test 2: Forensic confirmation in Hyperledger Besu of Block 81, recording legitimate client transaction `registerClientModel`.

5.6.3 Test 3: Ex-Post Auditing and Independent FedAvg Verifiability

To evaluate the third integrity property formalised in Section 3.3, we assessed the capability of an external auditor or regulator to retrospectively (*ex-post*) and autonomously verify the central server aggregation fidelity. This test verifies whether the coordinator computed FedAvg honestly without injecting unverified bias, omitting clients, or altering weights.

The audit was executed on Round 2 of a federated training run comprising two concurrent edge clients connected to Hyperledger Besu (contract `0x42699A76...`) and IPFS. Table 6 summarizes the complete chain of custody for Round 2, correlating sending accounts, transactions confirmed across consecutive blocks 73, 74, and 75, IPFS multihash identifiers, and execution latencies.

The audit protocol proceeded through the following steps:

1. **On-chain extraction and indexing:** In an off-band, decoupled manner, the auditor dynamically identifies participating clients $\mathcal{K}_2$ by querying `ModelUpdated` event logs for Round 2 in Besu (retrieving Client 2 and Client 3 accounts) and extracts their registered CIDs (`QmNh3TLS...` and `QmS5ViVf...`), along with the coordinator global model commitment (`QmWF7tJ1...`).
2. **Deterministic retrieval and reconstruction:** The auditor downloads all three binary model artifacts from IPFS (2.08 MB per model, totaling 6.24 MB retrieved in an accumulated 0.200 s [0.096 s, 0.044 s, and 0.060 s]) and executes canonical

Table 6 Chain of custody and cross-traceability on-chain/IPFS during Round 2 and ex-post audit.

Entity	Ethereum Address	Event / Method	Besu Block	Latency	IPFS CID	State / Result
Client 3	`0xf17f5215...`	`registerClient-Model`	Block 73	0.886 s	`QmS5ViVf...`	Confirmed on-chain
Client 2	`0x62730609...`	`registerClient-Model`	Block 74	4.311 s	`QmNh3TLS...`	Confirmed on-chain
Server	`0xfe3b557e...`	`updateModel` (FedAvg)	Block 75	4.709 s	`QmWF7tJ1...`	Global model committed
Auditor	*External (Off-chain)*	FedAvg Reconstruction	Query 73–75	0.339 s	`QmWF7tJ1...`	$\Delta = 0.000000$ **(MATCH)**

FedAvg aggregation:

$$\mathbf{w}_{\text{recon}}^{(2)} = \frac{1}{2}\mathbf{w}_{\text{cli2}}^{(2)} + \frac{1}{2}\mathbf{w}_{\text{cli3}}^{(2)} \tag{9}$$

generating the resulting model CID using the SHA-256 multihash algorithm.

3. **Tensor and cryptographic verification:** The reconstructed parameter tensors are compared layer-by-layer against the global model sealed by the coordinator in Block 75.

As documented in the console telemetry in Figure 13, the entire audit sequence (dynamic participant discovery, RPC query of commitments, downloading 6.24 MB from IPFS, tensor reconstruction, and verification across all 8 layers of the CNN) required only **0.3391 seconds**. Parametric discrepancy across all layers was strictly zero ($\Delta = 0.000000\text{e}+00$), resulting in an identical CID to that registered in Block 75 (`CID Equality: MATCH`), confirming aggregation correctness.

Furthermore, mechanism sensitivity was evaluated under simulated adversarial mode (`--simulate-tampering`). By injecting an additive perturbation of +0.1 into coordinator weights, the auditor detected the anomaly in **0.1805 seconds** ($\Delta = 1.000000\text{e}-01$), logging a formal discrepancy alert (`AUDIT FAILED: DISCREPANCY DETECTED`).

5.6.4 Synthesis of Validation and Security Scope

Table 7 synthesizes the quantitative metrics obtained across the three experimental integrity and auditing tests.

These experiments provide functional validation of the cryptographic primitives: when encountering altered in-transit parameters or uncommitted models, the protocol intercepts anomalies deterministically with response times in the order of tenths of a second (149 ms at the client, 31 ms at the coordinator, and 0.34 s in ex-post FedAvg auditing), preventing corrupt weights from contaminating training.

As delineated in our threat model (Section 3.4), operational boundaries must be emphasized: on-chain verification counters parameter tampering in transit, unauthorized submissions via ECDSA signatures, and rogue uncommitted models. However, if a malicious node generates poisoned updates from local training data (*data poisoning* or *label flipping*) [39] and legitimately signs the transaction in Besu, cryptographic commitments will match despite semantic contamination. Mitigating semantic

Table 7 Experimental validation of the integrity and auditing framework across manipulation and ex-post verification scenarios.

Test Scenario	**Defending / Auditing Entity**	**Preventive Action / Result**	**Latency**	**Weight Integrity**
In-Transit Tampering / Untrusted Coordinator	Client (Edge)	Deterministic interception / Preventive abort	0.1486 s	No local contamination
Unauthenticated Injection / Unregistered Client	Aggregation Coordinator	Deterministic interception / FedAvg exclusion	0.0309 s	Aggregation preserved
Ex-Post FedAvg Audit	Independent Auditor	Mathematical reconstruction / Faithful verification	0.34 s	Exact fidelity ($\Delta = 0$, CID MATCH)

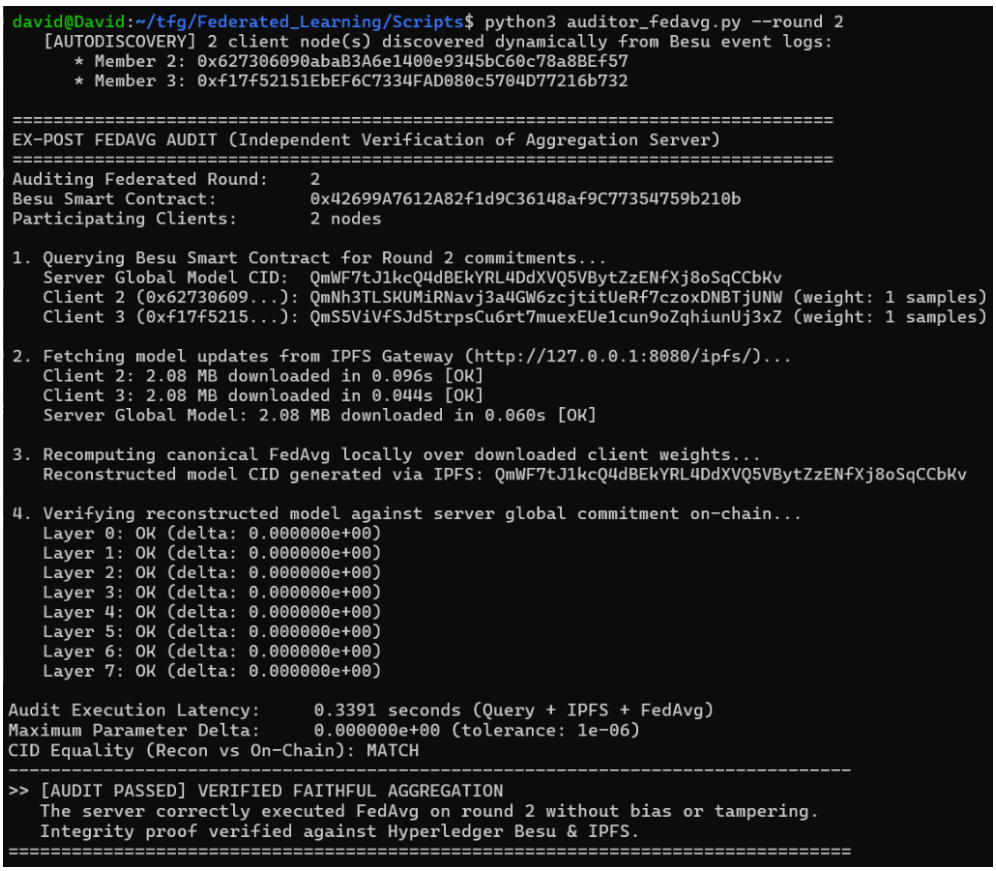


Fig. 13 Test 3: Independent auditor console log reconstructing Round 2 FedAvg aggregation from Besu and IPFS, verifying exact fidelity ($\Delta = 0.000000$) and CID match.

attacks requires augmenting traceability with Byzantine-robust aggregation rules (e.g., Krum [20], Median, or Trimmed-Mean) or conducting forensic ex-post auditing enabled by our architecture (Section 3.3).

5.7 EVM Gas Consumption and Predictability

Figure 14 illustrates EVM gas consumption for operations recorded across rounds.

The gas profile is invariant across all training iterations:

- `registerClientModel`: exactly **160,200 gas units**.
- `updateModel`: exactly **169,649 gas units**.

This consumption reflects the final version of the `FLRegistry` smart contract: incorporating the `onlyServer` access modifier into `updateModel` introduces an EVM identity check (`SLOAD` and conditional branch on `serverAddress`) that establishes execution cost at exactly 169,649 gas (a bounded overhead of 2,189 gas, +1.3% compared to an open contract without access control), remaining strictly constant and predictable

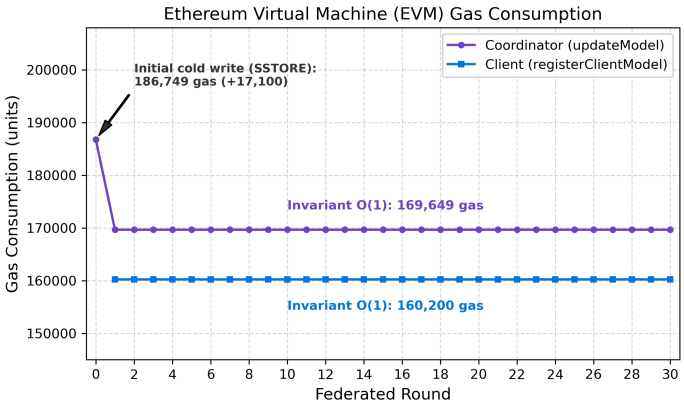


Fig. 14 Ethereum Virtual Machine gas consumption by transaction type across 30 federated rounds.

across all operational rounds. The sole observed exception occurs in Round 0 for the coordinator, which required **186,749 gas units**. This discrepancy (+17,100 gas) corresponds to the EVM `SSTORE` instruction when allocating uninitialized storage slots (*cold write*) and toggling the `hasModel` boolean flag.

This consumption invariance formally validates the decoupled architecture: by persisting only uniform-length strings on-chain (46-character IPFS CIDv0 identifiers encapsulating SHA-256 multihashes and bounded metadata), gas costs remain constant per transaction and independent of neural network parameter count and cumulative round count, guaranteeing predictable resource utilization in consortium networks.

5.8 On-Chain Immutability Audit and Forensic Traceability

To empirically verify bijective correspondence between Flower lifecycle events and immutable blockchain persistence, Table 8 presents a representative sample of transaction receipts extracted from runtime telemetry, correlated directly with the block explorer visualization (Figure 15).

Table 8 Sample of on-chain forensic traceability and audit receipts.

Block	Entity	Event / Method	Sender Address	Tx Hash Prefix	EVM Gas
8	Coordinator	Deploy `FLRegistry`	`0xfe3b557e...`	`0x939eb46c...`	1,999,000
13	Coordinator	`updateModel`	`0xfe3b557e...`	`0xa18e3bb4...`	169,649
14	Client 2	`registerClientModel`	`0x62730609...`	`0xc1c7402f...`	160,200
765	Coordinator	`updateModel` (Round 23)	`0xfe3b557e...`	`0x4ceabb7a...`	169,649
783	Coordinator	`updateModel` (Round 24)	`0xfe3b557e...`	`0xb9cb75f5...`	169,649
801	Coordinator	`updateModel` (Round 25)	`0xfe3b557e...`	`0xdf69295f...`	169,649



Fig. 15 On-chain immutability audit: Besu explorer visualization of `FLRegistry` contract deployment (Block 8) by coordinator (`0xfe3b55...`) alongside operational transactions 13 and 14.

Each block identifier and cryptographic transaction hash provides non-repudiable forensic proof, confirming the viability of the architecture for regulatory audits in vehicular and safety-critical distributed systems.

6 Conclusions and Future Work

This paper presented and empirically validated a comprehensive architecture designed to provide end-to-end traceability, cryptographic integrity, and independent auditability in Federated Learning, integrating Hyperledger Besu with IPFS and the Flower framework.

6.1 Main Conclusions

The empirical results lead to the following key conclusions:

- **Technical feasibility and manageable temporal overhead**: Integrating blockchain traceability into a realistic distributed deployment introduces an overhead of only **9.5%** (+6.34 s per round, increasing median cycle time from 66.62 s to 72.96 s), demonstrating that end-to-end auditability is practical for distributed AI without compromising operational training cadences.
- **Effectiveness and speed of bidirectional cross-verification**: Reliance on the central coordinator as a single point of trust is effectively mitigated. By cross-verifying IPFS CIDs against on-chain commitments, edge clients are protected against altered models (aborting in 0.149 s), while the coordinator immediately discards unauthenticated or tampered contributions (in 0.031 s), ensuring deterministic interception.
- **On-chain efficiency and predictability**: Decoupling model storage via IPFS guarantees an invariant EVM gas profile per transaction (160,200 gas for client registration and 169,649 gas for coordinator updates in steady state), avoiding EVM state bloat completely independent of neural network parameter count.

- **BFT consensus scalability**: Expanding validator nodes from 4 to 7 (elevating Byzantine fault tolerance from $f = 1$ to $f = 2$) increases transaction confirmation latency by only 0.10 s (100 ms) and median round time by just 0.39 s, overhead that is easily absorbed within local model computation.
- **Ex-post auditing and FedAvg verifiability**: The black-box limitation of centralized aggregation is eliminated through independent retrospective auditing. Mathematical FedAvg reconstruction was validated in 0.34 s with exact tensor fidelity ($\Delta = 0.000000$), detecting unauthorized parameter manipulations in 0.18 s without compromising private local training datasets.

6.2 Limitations and Future Work

Future research directions include:

1. **AI model optimization**: Because this study focused on system-level infrastructure, future work will integrate early stopping, advanced regularization (Dropout, BatchNorm), and deeper architectures to maximize domain classification performance.
2. **On-chain reputation and slashing mechanisms**: Leveraging the immutable contribution history to implement reputation-scoring algorithms and economic or logical slashing against persistently anomalous nodes.
3. **Advanced Byzantine aggregation**: Augmenting the framework with Byzantine-resilient aggregation rules (e.g., Krum [20], coordinate-wise trimmed means) coordinated directly via smart contracts.
4. **Non-IID distributions and V2X edge deployments**: Evaluating framework robustness under heterogeneous, non-IID client partitions and deploying nodes on vehicular edge hardware (such as NVIDIA Jetson platforms) over Cellular-V2X links.
5. **Hardware cryptographic key management**: Transitioning from software accounts to Hardware Security Modules (HSMs) or secure enclaves (e.g., Key Vault) for production-grade key custody.
6. **Decentralized governance and rotating committees**: Extending the `onlyServer` access modifier toward multi-signature DAO governance or rotating aggregation committees to decentralize global model authority across consortium stakeholders.

In summary, the evaluated architecture demonstrates that converging Blockchain with Federated Learning is not only viable, but establishes a sound methodological foundation for trustworthy, auditable artificial intelligence in safety-critical cooperative systems.

Declarations.

- **Funding**: Not applicable.
- **Conflict of interest**: The authors declare that they have no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

- **Data and materials availability**: The datasets and runtime telemetry logs generated during the empirical evaluation are available upon reasonable request.
- **Code availability**: Orchestration scripts, Solidity smart contracts, Docker Compose deployment templates, and independent audit modules are openly accessible at https://github.com/dpz6-ua/Blockchain_ML.

References

[1] McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) (2017)

[2] Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How to backdoor federated learning. In: Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 108, pp. 2938–2948. PMLR, ??? (2020). https://doi.org/10.48550/arXiv.1807.00459

[3] Chellapandi, V.P., Yuan, L., Wang, Z., Żak, S.H.: Federated learning for connected and automated vehicles: A survey of existing approaches and challenges. IEEE Transactions on Intelligent Vehicles **9**(1), 119–137 (2024) https://doi.org/10.1109/TIV.2023.3332675

[4] Qu, Y., Uddin, M.P., Gan, C., Xiang, Y., Gao, L., Yearwood, J.: Blockchain-enabled federated learning: A survey. ACM Computing Surveys **55**(4), 1–35 (2022) https://doi.org/10.1145/3524104

[5] Nakamoto, S.: Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf. Accedido en 2026 (2008)

[6] Zheng, Z., Xie, S., Dai, H.-N., Chen, X., Wang, H.: Blockchain challenges and opportunities: a survey. International Journal of Web and Grid Services **14**(4), 352–375 (2018) https://doi.org/10.1504/IJWGS.2018.095647

[7] Liu, S., Liu, L., Tang, J., Yu, B., Wang, Y., Shi, W.: Edge computing for autonomous driving: Opportunities and challenges. Proceedings of the IEEE **107**(8), 1697–1716 (2019) https://doi.org/10.1109/JPROC.2019.2915983

[8] Pujol, F.A., Mora, H., Ramírez, T., Rocamora, C., Bedón, A.: Blockchain-based framework for traffic event verification in smart vehicles. IEEE Access **12**, 9251–9266 (2024) https://doi.org/10.1109/ACCESS.2024.3352738

[9] CloudLab-AIA: FL-Blockchain: Federated Learning & Blockchain for Vehicular Event Recognition. https://github.com/cloudlab-aia/FL-Blockchain. Repositorio en GitHub. Accedido: 2026-06-15 (2026)

[10] Beutel, D.J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K.H., Parcollet, T., Gusmão, P.P.B., Lane, N.D.: Flower: A friendly federated learning research framework. arXiv preprint arXiv:2007.14390 (2020)

[11] National Institute of Standards and Technology: FIPS PUB 180-4: Secure Hash Standard (SHS). Technical report, U.S. Department of Commerce (2015)

[12] Buterin, V.: Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. https://ethereum.org/en/whitepaper/. Accedido en 2026 (2014)

[13] Hyperledger Besu: Hyperledger Besu Documentation. https://docs.besu-eth.org/private-networks. Accedido en 2026

[14] Moniz, H.: The istanbul bft consensus algorithm. arXiv preprint arXiv:2002.03613 (2020)

[15] Ethereum Foundation: Gas and fees. https://ethereum.org/en/developers/docs/gas/. Accedido en 2026

[16] Benet, J.: Ipfs - content addressed, versioned, p2p file system. arXiv preprint arXiv:1407.3561 (2014)

[17] Daniel, E., Tschorsch, F.: IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks. IEEE Communications Surveys & Tutorials **24**(1), 31–52 (2022) https://doi.org/10.1109/COMST.2022.3143147

[18] Trautwein, D., Raman, A., Tyson, G., Castro, I., Scott, W., Schubotz, M., Gipp, B., Psaras, Y.: Design and evaluation of IPFS: A storage layer for the decentralized web. In: Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22), pp. 739–752. ACM, ??? (2022). https://doi.org/10.1145/3544216.3544232

[19] Zhao, Y., Zhao, J., Jiang, L., Tan, R., Niyato, D., Li, Z., Lyu, L., Liu, Y.: Privacy-preserving blockchain-based federated learning for IoT devices. IEEE Internet of Things Journal **8**(3), 1817–1829 (2021) https://doi.org/10.1109/JIOT.2020.3017377

[20] Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 119–129 (2017)

[21] Kim, H., Park, J., Bennis, M., Debbah, M.: Blockchained on-device federated learning. IEEE Communications Letters **24**(6), 1279–1283 (2019) https://doi.org/10.1109/LCOMM.2019.2921755

[22] Weng, J., Weng, J., Zhang, J., Li, M., Zhang, Y., Luo, W.: Deepchain: Auditable and privacy-preserving deep learning with blockchain-based incentive. IEEE

Transactions on Dependable and Secure Computing **18**(5), 2438–2455 (2021) https://doi.org/10.1109/TDSC.2019.2952332

[23] Pokhrel, S.R., Choi, J.: Federated learning with blockchain for autonomous vehicles: Analysis and design challenges. IEEE Transactions on Communications **68**(8), 4734–4746 (2020) https://doi.org/10.1109/TCOMM.2020.2990686

[24] Chai, H., Leng, S., Chen, Y., Zhang, K.: A hierarchical blockchain-enabled federated learning algorithm for knowledge sharing in internet of vehicles. IEEE Transactions on Intelligent Transportation Systems **22**(7), 3975–3986 (2021) https://doi.org/10.1109/TITS.2020.3002712

[25] Lu, Y., Huang, X., Zhang, K., Maharjan, S., Zhang, Y.: Blockchain and federated learning for 5g beyond. IEEE Network **35**(1), 219–225 (2020) https://doi.org/10.1109/MNET.011.1900598

[26] Goh, E., Kim, D.-Y., Lee, K., Oh, S., Chae, J.-E., Kim, D.-Y.: Blockchain-enabled federated learning: A reference architecture design, implementation, and verification. IEEE Access **11**, 145747–145762 (2023) https://doi.org/10.1109/ACCESS.2023.3345360

[27] Bouacida, N., Mohapatra, P.: Vulnerabilities in federated learning. IEEE Access **9**, 63229–63249 (2021) https://doi.org/10.1109/ACCESS.2021.3075203

[28] Jere, M.S., Farnan, T., Koushanfar, F.: A taxonomy of attacks on federated learning. IEEE Security & Privacy **19**(2), 20–28 (2021) https://doi.org/10.1109/MSEC.2020.3046736

[29] Bao, X., Su, C., Xiong, Y., Huang, W., Xiang, Y.: FLChain: A blockchain for auditable federated learning with trust and incentive. In: Proceedings of the 5th International Conference on Big Data Computing and Communications (BIGCOM), pp. 151–159. IEEE, ??? (2019). https://doi.org/10.1109/BIGCOM.2019.00030

[30] Qi, Y., Hossain, M.S., Nie, J., Li, X.: Privacy-preserving blockchain-based federated learning for traffic flow prediction. Future Generation Computer Systems **117**, 328–337 (2021) https://doi.org/10.1016/j.future.2020.12.003

[31] Abou El Houda, Z., Moudoud, H., Brik, B., Khoukhi, L.: Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing. IEEE Transactions on Intelligent Transportation Systems **25**(7), 7661–7672 (2024) https://doi.org/10.1109/TITS.2024.3351699

[32] Yin, D., Chen, Y., Kannan, R., Bartlett, P.L.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 5650–5659. PMLR, ??? (2018)

[33] El Mhamdi, E.M., Guerraoui, R., Rouault, S.: The hidden vulnerability of distributed learning in byzantium. In: Proceedings of the 35th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 80, pp. 3521–3530. PMLR, ??? (2018)

[34] Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: Proceedings of the 28th Network and Distributed System Security Symposium (NDSS) (2021). https://doi.org/10.14722/ndss.2021.24434

[35] Xu, G., Li, H., Liu, S., Yang, K., Lin, X.: Verifynet: Secure and verifiable federated learning. IEEE Transactions on Information Forensics and Security **15**, 911–926 (2020) https://doi.org/10.1109/TIFS.2019.2929409

[36] Guo, X., Liu, Z., Li, J., Gao, J., Hou, B., Dong, C., Baker, T.: Verifl: Communication-efficient and fast verifiable aggregation for federated learning. IEEE Transactions on Information Forensics and Security **16**, 1736–1751 (2021) https://doi.org/10.1109/TIFS.2020.3043139

[37] Carletti, V., Foggia, P., Mazzocca, C., Parrella, G., Vento, M.: On the detectability of active gradient inversion attacks in federated learning. In: Proceedings of the 2026 IEEE Symposium on Security and Privacy (S&P), pp. 1931–1950. IEEE, ??? (2026). https://doi.org/10.1109/SP59124.2026.00085

[38] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshteyn, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 32, pp. 8024–8035 (2019)

[39] Tolpegin, V., Truex, S., Gursoy, M.E., Liu, L.: Data poisoning attacks against federated learning systems. In: Computer Security – ESORICS 2020. Lecture Notes in Computer Science, vol. 12308, pp. 480–501. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-58951-6_24